Домашнее задание № 1

Автоматизация задач при помощи Python — работа с файлами и данными

Учебный центр ШИФТ · Материал для слушателей

Практическая работа по семи разделам первого занятия: **TXT, словари Python, CSV, JSON, YAML, XML, преобразование и проверка данных**.

Обязательная часть — 4–6 часов суммарно. Задания со знаком ★ — повышенной сложности, выполняются **только по желанию** и **не влияют на зачёт**. Дополнительное время на них в оценку 4–6 часов не включено.

Время на выполнение

Раздел	Обязательная часть
1. TXT	
2. Словари Python	
3. CSV	
4. JSON	
5. YAML	
6. XML	
7. Преобразование и проверка данных	
Итого за задачи	

Как выполнять и сдавать

1. Создайте одну папку `homework_01` и внутри неё подпапки `01_txt`, `02_dict`, `03_csv`, `04_json`, `05_yaml`, `06_xml`, `07_validation`.
2. Каждый раздел **независимый**. Создайте указанный исходный файл в соответствующей подпапке; скопируйте его содержимое из задания. Программу сохраняйте в `main.py` той же подпапки. Результаты формируйте рядом с исходными данными.
3. Работайте с обычными `.py`-файлами и строковыми путями. Для файлов используйте `open()` и `with`; **`pathlib`, pandas, pickle не нужны**. Для YAML допустима одна внешняя библиотека **PyYAML**: `python -m pip install pyyaml`.
4. Запускайте каждый `main.py` из его подпапки, чтобы относительные имена файлов находились правильно. В PyCharm проверьте настройку **Working directory**, если получаете `FileNotFoundError`.
5. Сохраняйте исходные файлы без изменений. Все результаты записывайте в новые файлы; повторный запуск не должен дублировать данные.
6. Сдайте архив `homework_01.zip` с исходниками, программами и созданными отчётами. Файлы-задания необязательных пунктов ★ добавляйте, только если выполняли их.

Критерии приёмки: обязательные задачи выполнены; код запускается; чтение и запись корректны; расчёты совпадают с контрольными значениями; ошибки входных данных учитываются по условиям; исходные данные сохранены; результат можно воспроизвести повторным запуском.

1. TXT — анализ журнала событий

Создайте `01_txt/events.txt` в кодировке UTF-8:

```
2026-10-01;INFO;Сервис запущен
2026-10-01;ERROR;Не удалось загрузить отчёт
2026-10-02;INFO;Пользователь вошёл
2026-10-02;WARNING;Мало свободного места
2026-10-03;ERROR;Нет соединения с API
2026-10-03;INFO;Загрузка завершена
2026-10-04;ERROR;Ошибка подключения к базе
2026-10-04;WARNING;Долгий ответ сервера
```

Задача 1.1. Прочитать и подсчитать события

- Прочитайте файл с помощью `with open(..., encoding="utf-8")`.
- Каждая строка содержит `дата;уровень;сообщение`. Разделите строку на три части через `split(";", 2)`.
- Выведите количество всех событий и количество записей уровней `INFO`, `WARNING`, `ERROR`.
- Пустые строки, если они появятся, не считайте событиями.

Проверка: всего **8** событий; `INFO` — **3**, `WARNING` — **2**, `ERROR` — **3**.

Задача 1.2. Сформировать текстовый отчёт

- Создайте `errors.txt` только с ошибками уровня `ERROR` в виде `дата — сообщение`.
- Создайте `summary.txt` с количеством записей каждого уровня и общим числом событий.
- Для записи используйте `open(..., "w", encoding="utf-8")`. Повторный запуск должен заменять предыдущие отчёты, а не дописывать копии строк.

Проверка: в `errors.txt` должно быть **3** строки.

★ Задача 1.3. Поиск по дате — необязательно

Добавьте переменную `target_date = "2026-10-03"`. Выводите только события этой даты и их количество. Затем проверьте дату, которой нет в журнале: программа не должна завершаться с ошибкой.

2. Словари Python — статистика заказов

Создайте `02_dict/main.py` и поместите в него данные. Здесь **не нужно читать файл** — отрабатываем словари и списки словарей:

```
orders = [
    {"id": 101, "category": "Книги", "price": 500, "qty": 2, "status": "paid"},
    {"id": 102, "category": "Электроника", "price": 3200, "qty": 1, "status": "paid"},
    {"id": 103, "category": "Книги", "price": 700, "qty": 1, "status": "cancelled"},
    {"id": 104, "category": "Канцелярия", "price": 150, "qty": 3, "status": "paid"},
    {"id": 105, "category": "Книги", "price": 800, "qty": 1, "status": "paid"},
    {"id": 106, "category": "Канцелярия", "price": 200, "qty": 2, "status": "paid"},
]
```

Задача 2.1. Подсчёт оплаченных заказов — обязательно

- Переберите список циклом `for`; учитывайте только записи со статусом `paid`.
- Для каждого оплаченного заказа вычислите стоимость `price * qty`.
- Выведите количество оплаченных заказов и общую выручку. В коде используйте обращение к словарю по ключу и хотя бы один вызов `get()`.

Проверка: 5 оплаченных заказов, общая выручка **5850 руб.**

Задача 2.2. Группировка по категориям — обязательно

- Создайте словарь, где ключ — категория, а значение — общая выручка этой категории.
- Накопление сумм реализуйте через `get()` или `setdefault()`; используйте `items()` для вывода результата.
- Не учитывайте отменённый заказ.

Проверка: `Книги` — **1800**, `Электроника` — **3200**, `Канцелярия` — **850**.

★ Задача 2.3. Вложенная статистика — необязательно

Для каждой категории храните **и сумму, и количество оплаченных заказов**: `{"Книги": {"amount": 1800, "count": 2}, ...}`. Выведите категорию с наибольшей выручкой. Используйте обычные циклы, без сторонних библиотек.

3. CSV — обработка заказов магазина

Создайте `03_csv/orders.csv` с разделителем **точка с запятой**:

```
id;product;category;price;quantity;status
1;Папка;Канцелярия;120;3;paid
2;Клавиатура;Электроника;2500;1;paid
3;Тетрадь;Канцелярия;80;5;paid
4;Мышь;Электроника;900;1;cancelled
5;Ручка;Канцелярия;40;10;paid
6;Кабель;Электроника;350;2;paid
```

Задача 3.1. Прочитать и отфильтровать CSV

- Используйте `csv.DictReader` и `delimiter=";"`; откройте файл с `newline=""`.
- Учтите, что значения CSV — строки: преобразуйте `price` и `quantity` в `int`.
- Оставьте только записи со статусом `paid`; рассчитайте поле `total = price * quantity`.
- Выведите на экран название товара и его итоговую стоимость.

Проверка: 5 оплаченных записей; сумма `total` — **4360 руб.**

Задача 3.2. Сохранить два отчёта

- С помощью `csv.DictWriter` сохраните `paid_orders.csv` с полями `id;product;category;total`.
- Сохраните `category_summary.csv` с полями `category;amount`: суммарная выручка оплаченных заказов по категориям.
- Оба выходных файла используйте с `delimiter=";"`, `encoding="utf-8"`, `newline=""`.

Проверка: `Канцелярия` — **1160 руб.**, `Электроника` — **3200 руб.**; в первом отчёте **5** строк данных плюс заголовок.

★ Задача 3.3. Испорченная строка — необязательно

Добавьте в исходный CSV строку `7;Монитор;Электроника;неизвестно;1;paid`. Измените программу так, чтобы она не падала при преобразовании цены: ошибочная строка должна попасть в `csv_errors.txt`, остальные — обработаться как прежде. Сохраните причину ошибки.

4. JSON — клиенты и их покупки

Создайте `04_json/clients.json`:

```
{
  "clients": [
    {"id": 1, "name": "Анна", "city": "Казань", "orders": [1200, 800]},
    {"id": 2, "name": "Олег", "city": "Тула", "orders": [500]},
    {"id": 3, "name": "Мария", "city": "Казань", "orders": []},
    {"id": 4, "name": "Илья", "city": "Пермь", "orders": [1500, 500, 300]}
  ]
}
```

Задача 4.1. Прочитать вложенные данные

- Используйте `json.load()` для чтения файла через `open()`.
- Получите список клиентов из ключа `clients` и выведите имя, город и сумму покупок каждого клиента.
- Для клиента без покупок выводите **0**, не пропускайте его.

Проверка: Анна — **2000**, Олег — **500**, Мария — **0**, Илья — **2300**.

Задача 4.2. Сформировать JSON-отчёт

- Подсчитайте общее количество клиентов, количество отдельных покупок (элементов в списках `orders`) и сумму всех покупок.
- Сформируйте словарь `by_city`, содержащий количество клиентов по каждому городу.
- Сохраните `report.json` через `json.dump(..., ensure_ascii=False, indent=2)` со следующими ключами: `total_clients`, `total_orders`, `total_amount`, `clients_without_orders`, `by_city`.

Проверка: `total_clients` — **4**, `total_orders` — **6**, `total_amount` — **4800**, `clients_without_orders` — `["Мария"]`, в `Казани` — **2** клиента.

★ Задача 4.3. Рейтинг покупателей — необязательно

Создайте `top_clients.json`: отсортируйте клиентов по сумме покупок по убыванию, запишите имя и итоговую сумму. Если суммы одинаковые, допускается любой порядок таких клиентов.

5. YAML — настройки без правки Python-кода

В этом разделе установите **PyYAML** командой `python -m pip install pyyaml`, затем создайте `05_yaml/settings.yaml`:

```
report:
  statuses:
    - paid
    - shipped
  min_amount: 1000
  output_file: selected_orders.txt
  currency: RUB
```

В `05_yaml/main.py` создайте список:

```
orders = [
    {"id": 1, "amount": 1200, "status": "paid"},
    {"id": 2, "amount": 750, "status": "paid"},
    {"id": 3, "amount": 1600, "status": "shipped"},
    {"id": 4, "amount": 2000, "status": "cancelled"},
    {"id": 5, "amount": 1000, "status": "paid"},
]
```

Задача 5.1. Прочитать YAML-конфигурацию

- Загрузите файл через `yaml.safe_load()` и `open()`.
- Получите из `report` допустимые статусы, минимальную сумму, имя выходного файла и валюту.
- Выведите прочитанные настройки. Проверьте, что статусы представлены списком, а минимальная сумма — числом.

Задача 5.2. Применить настройки к заказам

- Отберите заказы, у которых статус входит в `statuses`, а `amount` **больше или равен** `min_amount`.
- Сохраните выбранные заказы в файл, имя которого берётся из `output_file`; добавьте итоговое количество и общую сумму с валютой.
- Поменяйте в YAML `min_amount` на `1500` и повторите запуск **без изменения Python-кода**.

Проверка: при пороге **1000** отобрано **3** заказа на **3800 RUB**. При пороге **1500** — **1** заказ на **1600 RUB**.

★ Задача 5.3. Сохранить настройки — необязательно

Создайте в программе словарь с новым набором настроек и запишите его в `settings_new.yaml` через `yaml.safe_dump()`. Сохраните кириллицу читаемой (`allow_unicode=True`) и вложенную структуру в блочном формате (`default_flow_style=False`).

6. XML — каталог товаров и остатки

Создайте `06_xml/catalog.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<catalog>
  <product id="A-01" category="office">
    <name>Бумага</name><price>300</price><stock>6</stock>
  </product>
  <product id="A-02" category="office">
    <name>Папка</name><price>150</price><stock>4</stock>
  </product>
  <product id="B-01" category="devices">
    <name>Мышь</name><price>900</price><stock>2</stock>
  </product>
  <product id="B-02" category="devices">
    <name>Клавиатура</name><price>2000</price><stock>0</stock>
  </product>
</catalog>
```

Задача 6.1. Прочитать дерево XML

- Используйте `xml.etree.ElementTree`: `ET.parse()`, `getroot()`, `findall("product")`.
- Для каждого товара выведите `id` и `category` из атрибутов, а название, цену и остаток — из вложенных элементов.

- Преобразуйте цену и остаток в числа, рассчитайте стоимость каждого остатка: `price * stock`.

Проверка: 4 товара, общий остаток **12** штук, общая стоимость остатков **4200 руб.**

Задача 6.2. Создать новый XML и краткий отчёт

- В новый `available_products.xml` запишите только товары с остатком **больше нуля**. Сохраняйте `id`, название, цену и остаток; можно добавить категорию.
- Создайте `inventory.txt` с количеством всех товаров, количеством товаров в наличии и общей стоимостью остатков.
- Исходный `catalog.xml` не изменяйте. Для создания нового документа можно использовать `ET.Element()`, `ET.SubElement()` и `ElementTree.write()`.

Проверка: в новом XML — **3** товара. В `inventory.txt` — **4** товара всего, **3** позиции в наличии, стоимость **4200 руб.**

★ Задача 6.3. Обновить остаток — необязательно

Для товара `A-01` уменьшите остаток на 2, затем сохраните полный каталог как `updated_catalog.xml`. Перед изменением проверьте, что остаток не станет отрицательным. Повторно прочитайте созданный XML и убедитесь, что новый остаток — **4**.

7. Преобразование и проверка данных — очистка регистраций

Создайте `07_validation/registrations.csv` с разделителем `;`:

```
id;name;age;email;status
1;Анна;23;anna@example.org;confirmed
2;Олег;17;oleg@example.org;confirmed
3;Ирина;29;irina.example.org;confirmed
4;Мария;34;maria@example.org;cancelled
5;Анна;23;anna@example.org;confirmed
6; ;41;empty@example.org;confirmed
7;Павел;not_a_number;pavel@example.org;confirmed
8;Елена;30;elena@example.org;confirmed
9;Игорь;19;igor@example.org;confirmed
```

Задача 7.1. Проверить поля и отсеять ошибки

- Прочитайте CSV через `csv.DictReader`, затем удалите лишние пробелы у текстовых полей с помощью `strip()`.
- Проверьте условия: `id` — положительное целое число; `name` — непустое; `age` — целое число от **18 до 80** включительно; `status` — `confirmed` или `cancelled`.
- Для учебной проверки email достаточно убедиться, что есть `@`, непустая часть до неё и хотя бы одна точка в части после `@`. Это **не полноценная проверка существования email-адреса**.
- Найдите повторяющиеся email без учёта регистра (`lower()`), если запись прошла остальные проверки. Повторный email считайте ошибкой.
- Для преобразований используйте `try/except ValueError`, а не просто проверку, что строка состоит из цифр.

Проверка: 9 строк данных, **4** корректные, **5** ошибочных. Ошибки: возраст меньше 18, неправильный email, повторный email, пустое имя, некорректный возраст.

Задача 7.2. Сохранить корректные записи и ошибки

- Создайте `clean_registrations.csv` с теми же заголовками и только корректными строками.

- Создайте `errors.csv` с колонками `row_number;id;reason`. `row_number` — **номер строки исходного CSV, включая заголовок** (первой записи соответствует строка 2).
- Создайте `summary.json` с полями `total`, `valid`, `invalid`, `confirmed`, `cancelled`. Последние два показателя вычисляйте **только по корректным записям**.
- Не заменяйте исходный файл. При повторном запуске все три отчёта должны обновляться без дублирования записей.

Проверка: `total` — **9**, `valid` — **4**, `invalid` — **5**, `confirmed` — **3**, `cancelled` — **1**; в `errors.csv` — **5** строк данных плюс заголовок.

★ Задача 7.3. Гибкие правила проверки — необязательно

Вынесите допустимый возраст и статусы в `rules.json` и получайте их через `json.load()`. Проверьте программу после изменения максимального возраста с 80 до 30: запись Марии (34 года) должна перейти в список ошибок. Дополнительно можно собирать **все причины ошибок одной строки**, а не только первую.